

Assistente Médico LangGraph

Documentação Técnica do Projeto

Sistema de apoio à decisão clínica com LangGraph, PostgreSQL + pgvector e RAG

Tech Challenge – Fase 3 · PosTech IADT (9IADT)

Repositório: github.com/MacielMjose/assistente-medico-langgraph

Última atualização: 2026-09-13

1. Visão Geral

O Assistente Médico LangGraph é um sistema de apoio à decisão clínica que combina orquestração de fluxos com LangGraph, armazenamento vetorial em PostgreSQL com a extensão pgvector, e uma arquitetura de RAG (Retrieval-Augmented Generation) para enriquecer as respostas do assistente com fontes de condutas médicas. O sistema foi projetado com suporte a múltiplos provedores de LLM, permitindo alternar entre modelos locais (via Ollama) e outros provedores.

O projeto integra o Tech Challenge da Fase 3 do PosTech IADT (turma 9IADT), cujo escopo envolve o fine-tuning de uma LLM com dados médicos internos e a construção de um pipeline com LangChain/LangGraph, respeitando limites de atuação (o assistente nunca prescreve sem validação humana), logging para auditoria e explainability das respostas.

2. Objetivos e Requisitos do Desafio

Principais requisitos definidos para o projeto:

- Fine-tuning de uma LLM (LLaMA, Falcon ou similar) com dados médicos internos: protocolos, FAQs de médicos, laudos e receitas.
- Pipeline com LangChain para consulta a bases estruturadas e contextualização com dados do paciente.
- Fluxos de decisão orquestrados com LangGraph.
- Guardrails de segurança: o assistente nunca prescreve sem validação humana.
- Logging detalhado para fins de auditoria.
- Explainability das respostas geradas.
- RAG para enriquecer o contexto com fontes de condutas médicas (livros e literatura), extraídas via LangChain a partir de documentos em Excel, PDF etc.

Entregáveis esperados:

- Repositório Git com o pipeline de fine-tuning e a integração LangChain/LangGraph.
- Dataset anonimizado ou sintético.
- Relatório técnico.
- Vídeo demonstrativo de até 15 minutos.

Datasets sugeridos como referência: PubMedQA e MedQuAD.

3. Arquitetura e Estrutura do Projeto

A estrutura de diretórios do repositório é organizada da seguinte forma:

•	
— main.py	# Ponto de entrada (workflow LangGraph)
— requirements.txt	# Dependências Python
— .env.sample	# Template de variáveis de ambiente
— src/	
— models/	# Modelos Pydantic (estado, dados)
— services/	
— llm_provider_service.py	# Gerenciamento de provedores LLM
— db/	
— connection.py	# Conexão PostgreSQL
— models.py	# Modelos SQLAlchemy
— repos/	# Repositórios de dados
— buscar/	# Busca vetorial (RAG)
— embeddings.py	# Provedores de embeddings
— routes/	# Endpoints da API (se aplicável)

```
├── database/
│   ├── etl_seed.py           # Seed de dados iniciais
│   ├── embeddings_ingest.py  # Ingestão de conhecimento (RAG)
│   └── migrations/          # Scripts de migração
├── tests/                    # Testes automatizados
└── knowledge/
    ├── pdf/                  # Documentos (protocolos, diretrizes)
    └── excel/                # Planilhas de referência
```

Componentes principais:

- `main.py` — ponto de entrada que dispara o workflow orquestrado pelo LangGraph.
- `src/services/llm_provider_service.py` — camada de abstração para múltiplos provedores de LLM.
- `src/db` — conexão, modelos SQLAlchemy, repositórios e busca vetorial (RAG) sobre o PostgreSQL/pgvector.
- `database/embeddings_ingest.py` — ingestão do conhecimento médico (PDF, Excel) para a base vetorial.
- `knowledge/` — base de documentos de referência (protocolos, diretrizes, planilhas).

4. Fluxo de Decisão do Assistente (LangGraph)

O diagrama a seguir representa o grafo de execução construído com LangGraph em main.py, mostrando a sequência de nós percorrida para cada atendimento a um paciente, bem como os pontos de decisão e os desvios para tratamento de erro/contexto inválido.

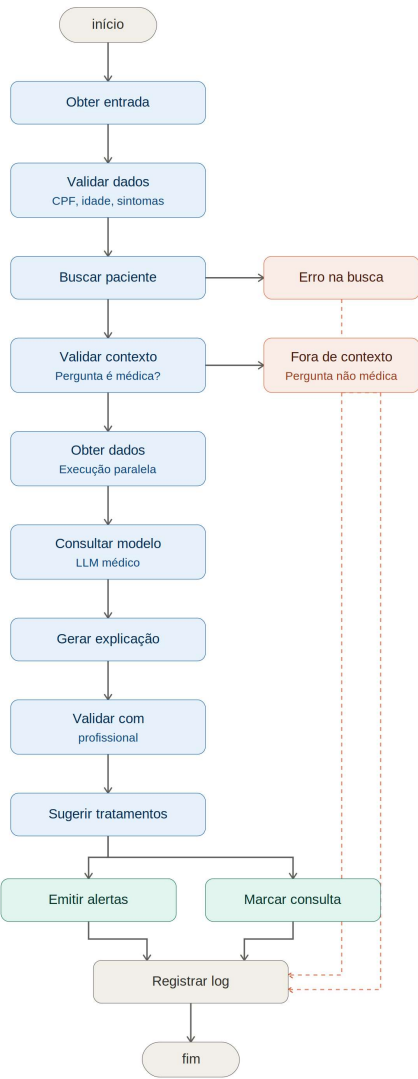


Figura 1 — Fluxo LangGraph do assistente médico: entrada, validações, busca de paciente, consulta ao modelo LLM, geração de explicação, validação profissional, sugestão de tratamentos, tratamento de erros e registro de auditoria.

4.1 Etapas e decisões tomadas durante a execução

Nó	O que acontece / decisão tomada
Obter entrada	Recebe os dados de entrada da requisição do atendimento (identificação do paciente e a pergunta/caso a ser avaliado), dando início ao estado do grafo.
Validar dados	Valida os dados básicos do paciente — CPF, idade e sintomas informados — antes de prosseguir. Dados inconsistentes interrompem o fluxo normal do atendimento.
Buscar paciente	Consulta o prontuário do paciente na base estruturada. Decisão: se o paciente não é localizado, o fluxo desvia para o nó "Erro na busca", que trata a falha e encaminha diretamente para o registro de auditoria, encerrando o atendimento sem prosseguir para a análise clínica.

Validar contexto	Verifica se a pergunta recebida está dentro do escopo médico. Decisão: perguntas fora de contexto (não médicas) são desviadas para o nó "Fora de contexto", que registra o motivo e encaminha para auditoria, sem consultar o modelo de LLM.
Obter dados (execução paralela)	Uma vez validado o contexto, o grafo busca em paralelo as informações complementares necessárias (histórico, exames, protocolos/RAG) para enriquecer a consulta ao modelo.
Consultar modelo (LLM médico)	Envia o contexto consolidado (dados do paciente + pergunta + conhecimento recuperado) ao modelo de LLM fine-tuned/Ollama para gerar uma resposta clínica preliminar.
Gerar explicação	Constrói a explicação/justificativa da resposta do modelo, dando suporte à explainability exigida pelo projeto (rastreabilidade da origem da informação).
Validar com profissional	Aplica o guardrail central do sistema: a resposta é marcada como sugestão sujeita à validação humana, nunca como prescrição definitiva.
Sugerir tratamentos	A partir da resposta validada, o grafo se ramifica em duas ações que podem ocorrer de forma complementar: "Emitir alertas" (ex.: risco identificado, conflito de alergia) e "Marcar consulta" (encaminhamento para acompanhamento presencial).
Emitir alertas / Marcar consulta	Ambos os caminhos convergem novamente antes do encerramento, garantindo que qualquer alerta emitido e qualquer agendamento gerado sejam registrados no mesmo log de auditoria.
Registrar log (auditoria)	Nó de convergência final: recebe tanto o caminho "feliz" (sugestão de tratamento) quanto os desvios de erro ("Erro na busca" e "Fora de contexto", indicados no diagrama pelas linhas tracejadas), registrando todos os acessos e decisões para fins de auditoria antes de encerrar o atendimento.

Em resumo, o grafo aplica três pontos de decisão críticos para cada paciente: (1) o paciente existe na base? (2) a pergunta está dentro do escopo médico? e (3) a resposta gerada pelo modelo precisa de alerta e/ou encaminhamento para consulta. Qualquer desvio de erro ou de escopo é registrado em auditoria da mesma forma que o fluxo completo, garantindo rastreabilidade de todas as interações — inclusive as que não chegam a acionar o modelo de LLM.

5. Pré-requisitos

- Python 3.10 ou superior
- Docker e Docker Compose
- Ollama (local ou via Docker), para execução de modelos sem dependência de nuvem

6. Instalação e Configuração

6.1 Clonar e configurar o ambiente

```
git clone <repo-url>
cd assistente-medico-langgraph

# Instale as dependências
pip install -r requirements.txt

# Configure as variáveis de ambiente
cp .env.sample .env
```

6.2 Subir PostgreSQL e Ollama (opcional)

```
docker compose up -d
```

6.3 Migrações e seed de dados

```
python database/connection.py # Testa a conexão
python database/etl_seed.py   # Popula dados iniciais
```

7. Configuração do Ollama

O projeto utiliza Ollama local para executar modelos de chat e de embeddings sem dependências de nuvem.

Pré-requisitos específicos:

- Docker Compose rodando (serviço "ollama")
- Modelo de chat importado no Ollama (ex.: assistente_medico_small)
- Modelo de embeddings (ex.: bge-m3)

Variáveis de ambiente (.env):

```
MEDPT_OLLAMA_BASE_URL=http://localhost:11434
MEDPT_OLLAMA_CHAT_MODEL=assistente_medico_small
MEDPT_EMBEDDING_PROVIDER=ollama
MEDPT_OLLAMA_EMBEDDING_MODEL=bge-m3
```

Importando os modelos:

```
# Baixar modelo de embeddings
ollama pull bge-m3

# Importar modelo customizado (se tiver Modelfile e arquivos GGUF)
# 1. Coloque seu Modelfile no diretório correto
# 2. Execute
ollama create assistente_medico_small -f /caminho/Modelfile

# 3. Verifique
ollama list
```

Exemplo de Modelfile:

```
FROM /caminho/seu-modelo-base.gguf

# Parâmetros opcionais
PARAMETER temperature 0.7
PARAMETER top_p 0.9
PARAMETER top_k 40
```

8. Executando o Assistente

```
# Certifique-se de que Ollama está rodando
ollama serve

# Em outro terminal, execute
python main.py
```

9. Segurança e Conformidade

O assistente implementa guardrails para uso responsável em contexto clínico:

- Validação de Contexto Médico — bloqueia perguntas fora do escopo de saúde.
- Verificação de Alergias — detecta conflitos entre medicamentos e alergias registradas.
- Disclaimer Automático — sempre inclui aviso de que a análise é apoio à decisão, não prescrição.

- Auditoria — todos os acessos são registrados em log.

10. Testes

```
# Rodar testes unitários
pytest tests/ -v

# Com testes de integração (PostgreSQL)
export MEDPT_TESTAR_PG=1
pytest tests/ -v
```

11. Stack Tecnológica

Componente	Tecnologia
Orquestração de fluxo	LangGraph
Camada de recuperação (RAG)	LangChain + PostgreSQL/pgvector
Modelos de LLM / Embeddings	Ollama (local), suporte a múltiplos provedores
Persistência	PostgreSQL (via SQLAlchemy)
Base de conhecimento	Documentos PDF e planilhas Excel (knowledge/)
Infraestrutura local	Docker / Docker Compose
Testes	pytest

12. Referências

- LangGraph — <https://langchain-ai.github.io/langgraph/>
- Ollama — <https://ollama.ai/>
- LangChain Ollama — <https://python.langchain.com/docs/integrations/llms/ollama/>

13. Contato

Para dúvidas ou problemas relacionados ao projeto, abra uma issue no repositório oficial no GitHub.